

BÁO CÁO BÀI TẬP LỚN: CHƯƠNG TRÌNH CỜ CARO AI

- Môn học: Cơ sở Trí tuệ Nhân tạo
- Họ và tên sinh viên: Nguyễn Tiến Phương
- Mã số sinh viên (MSSV): 24022430

1. MÔ TẢ BÀI TOÁN CỜ CARO VÀ LUẬT CHƠI

1.1.

Bài toán

Bài toán yêu cầu xây dựng một chương trình máy tính cho phép người chơi con người tương tác và thi đấu cờ Caro với máy tính (AI Agent). Máy tính sẽ đóng vai trò là một thực thể thông minh, sử dụng các thuật toán tìm kiếm có đối thủ kết hợp hàm đánh giá trạng thái nhằm tự động tính toán và đưa ra nước đi tối ưu nhất trong thời gian thực.

1.2.

Luật chơi và điều kiện thắng

- Kích thước bàn cờ: Bàn cờ được cấu hình với kích thước ma trận vuông 9x9 (thỏa mãn yêu cầu tối thiểu của đề bài).
- Cơ chế đi quân: Hai bên (Người và Máy) luân phiên nhau đánh quân vào các ô trống trên bàn cờ. Khi một ô đã có quân cờ thì không được phép đánh đè lên.
- Ký hiệu: Người chơi được ký hiệu là quân X, Máy tính được ký hiệu là quân O, các ô trống được ký hiệu bằng dấu chấm .
- Điều kiện thắng: Bên nào tạo thành một chuỗi gồm 4 quân liên tiếp theo hàng ngang, hàng dọc, đường chéo xuôi hoặc đường chéo ngược trước sẽ giành chiến thắng chung cuộc.
- Quy định đặc biệt: Bài toán không xét luật chặn hai đầu (chỉ cần tạo đủ chuỗi 4 quân là thắng ngay lập tức).
- Điều kiện hòa: Nếu bàn cờ đã được lấp đầy tất cả các ô trống mà vẫn không có bên nào đạt được chuỗi 4 quân liên tiếp thì trận đấu kết thúc với kết quả Hòa.

2. THIẾT KẾ HỆ THỐNG VÀ CẤU TRÚC DỮ LIỆU

2.1.

Cách biểu diễn trạng thái bàn cờ

Để biểu diễn cấu trúc không gian trạng thái trò chơi, chương trình sử dụng cấu trúc mảng hai chiều (ma trận) kích thước 9x9. Mỗi phần tử trong ma trận tại tọa độ (dòng, cột) quản lý một trong ba trạng thái chuỗi ký tự hằng số:

- '.' : Đại diện cho ô trống, người chơi có thể hạ quân.

- 'X' : Đại diện cho quân cờ của Người chơi (Đại diện cho tầng cực tiểu hóa điểm - MIN).
- 'O' : Đại diện cho quân cờ của Máy tính (Đại diện cho tầng cực đại hóa điểm - MAX).

2.2.

Cách sinh nước đi hợp lệ

- Giải pháp vét cạn thông thường: Duyệt qua toàn bộ các ô có ký hiệu '.' trên bàn cờ. Tuy nhiên, phương pháp này làm hệ số nhánh (Branching Factor) phình to, dẫn đến bùng nổ tổ hợp không gian trạng thái khi độ quy sâu.
- Giải pháp tối ưu hóa đã cài đặt: Để thu hẹp phạm vi tìm kiếm hiệu quả, chương trình áp dụng kỹ thuật giới hạn vùng lân cận. Hàm sinh nước đi sẽ chỉ quét và thu thập các ô trống nằm trong bán kính 2 ô xung quanh tất cả các quân cờ (X hoặc O) hiện đang có trên bàn cờ.
- Khai cuộc đặc biệt: Tại trạng thái đầu ván khi bàn cờ hoàn toàn trống, hàm sinh nước đi sẽ mặc định trả về tọa độ trung tâm là ô (4, 4) để định hình thế trận đối xứng và tối ưu điểm chiến lược ban đầu.

2.3.

Cách kiểm tra trạng thái kết thúc

Chương trình cài đặt hai hàm kiểm tra độc lập nhằm xác định điểm ngắt của trò chơi:

- Hàm `check_win(board, player)`: Nhận vào ma trận hiện tại và ký hiệu người chơi cần kiểm tra. Hàm tiến hành quét toàn bộ bàn cờ; tại mỗi vị trí có quân của người chơi đó, hàm sẽ kiểm tra chuỗi tịnh tiến tăng dần 4 ô liên tiếp theo 4 hướng (Ngang (0, 1), Dọc (1, 0), Chéo xuôi (1, 1), Chéo ngược (1, -1)). Nếu có hướng nào đạt đủ 4 quân, hàm lập tức trả về True.
- Hàm `is_board_full(board)`: Quét toàn bộ ma trận, nếu không còn tồn tại bất kỳ ký tự '.' nào, hàm trả về True (Bàn cờ đầy, kích hoạt trạng thái Hòa).

3. THUẬT TOÁN TÌM KIẾM VÀ HÀM ĐÁNH GIÁ TRẠNG THÁI

3.1.

Thuật toán Minimax đã cài đặt (Level 1)

Thuật toán Minimax được xây dựng theo mô hình cây trò chơi hai người, tổng bằng không. Nó hoạt động dựa trên cơ chế đệ quy và quay lùi (Backtracking) để duyệt không gian trạng thái với cấu trúc cụ thể:

- Điều kiện dừng cơ sở: * Nếu trạng thái giả định dẫn đến Máy thắng: Trả về giá trị điểm kết thúc tối đa là (1.000.000 + độ_sâu).

- Nếu trạng thái giả định dẫn đến Người thắng: Trả về giá trị điểm kết thúc tối thiểu là $(-1.000.000 - \text{độ_sâu})$.
- Nếu chạm giới hạn độ sâu tìm kiếm ($\text{depth} == 0$) hoặc hòa cờ: Gọi hàm Heuristic để tính toán điểm số ước lượng của trạng thái đó.
- Tầng MAX (Lượt của Máy): Duyệt qua các nước đi hợp lệ, giả định đặt quân O, gọi đệ quy xuống tầng dưới và chọn lấy giá trị điểm số Lớn nhất (MAX) nhằm tối đa hóa cơ hội thắng.
- Tầng MIN (Lượt của Người): Duyệt qua các nước đi hợp lệ, giả định đặt quân X, gọi đệ quy xuống tầng dưới và chọn lấy giá trị điểm số Nhỏ nhất (MIN) dựa trên giả thuyết đối thủ luôn đi nước cờ thông minh nhất để ép điểm AI xuống.
- Kết quả trả về: Hàm trả về một bộ đôi gồm: (nước_đi_tốt_nhất , $\text{giá_trị_đánh_giá_tương_ứng}$).

3.2.

Thuật toán Cắt nhánh Alpha-Beta đã cài đặt (Level 2)

Thuật toán Alpha-Beta Pruning được phát triển trực tiếp dựa trên cấu trúc đệ quy của Minimax nhằm tối ưu hóa hiệu năng bằng cách bỏ sung hai lần ranh động trong suốt quá trình duyệt cây:

- α (Alpha): Đại diện cho giá trị lựa chọn tốt nhất hiện tại (lớn nhất) mà tầng MAX có thể bảo đảm vững chắc. Giá trị khởi tạo là $-\infty$.
- β (Beta): Đại diện cho giá trị lựa chọn tốt nhất hiện tại (nhỏ nhất) mà tầng MIN có thể bảo đảm vững chắc. Giá trị khởi tạo là $+\infty$.
- Cơ chế cắt nhánh: Tại bất kỳ nút nào trên cây tìm kiếm, sau khi cập nhật các giá trị α và β , nếu phát hiện điều kiện $\beta \leq \alpha$, thuật toán sẽ lập tức bẻ gãy vòng lặp (break) và dừng việc duyệt toàn bộ các nhánh con còn lại. Lý do vì các nhánh này chắc chắn chứa những nước đi tệ hơn các phương án tối ưu mà một trong hai bên đã tìm được trước đó, nên việc duyệt tiếp là vô nghĩa.

Để bảo đảm tính công bằng khi đánh giá hiệu năng thực nghiệm, thuật toán Alpha-Beta được cấu hình sử dụng chung cùng một hàm đánh giá Heuristic và chạy trên cùng một độ sâu giới hạn giống hệt như Minimax.

3.3.

Hàm đánh giá trạng thái (Heuristic)

Do không gian cờ Caro quá lớn, không thể duyệt cây đến tận cùng ván đấu, hàm Heuristic đóng vai trò đo lường "sức mạnh" của một trạng thái tĩnh. Hàm thực hiện quét toàn bộ ma trận theo các cửa sổ gồm chuỗi 4 ô liên tiếp theo cả 4 hướng (Ngang, Dọc, Chéo xuôi, Chéo ngược).

Điểm số tổng thể của bàn cờ được tính bằng tổng điểm thể trận của Máy tính (quân O) trừ đi tổng điểm thể trận của Người chơi (quân X). Quy tắc chấm điểm cho từng chuỗi 4 ô cụ thể như sau:

- Chuỗi chứa 4 quân O: Trả về +100.000 điểm (Thế cờ thắng tuyệt đối).
- Chuỗi chứa 3 quân O và 1 ô trống: Trả về +5.000 điểm (Cơ hội tấn công cực cao, chuẩn bị hạ đường 4 thắng).
- Chuỗi chứa 2 quân O và 2 ô trống: Trả về +200 điểm (Thế trận tích lũy, phát triển chiến thuật).
- Chuỗi chứa 3 quân X và 1 ô trống: Trả về -4.000 điểm. *Ý nghĩa:* Đây là hình phạt nặng nề khi phát hiện đối thủ sắp đạt hàng 4 thắng trận. Điểm phạt này sẽ ép tăng MAX phải hạ quân vào ô trống đó để thực hiện phòng thủ, chặn khẩn cấp nước đi của người chơi.

4. KẾT QUẢ THỰC NGHIỆM VÀ PHÂN TÍCH (LEVEL 3)

Hệ thống tiến hành thực nghiệm tự động bằng cấu hình script benchmark.py trên chuẩn xác 6 kịch bản thế cờ khác nhau. Dữ liệu đo đặc chi tiết về số trạng thái đã duyệt và thời gian chạy (ms) ở các độ sâu 1, 2, 3 được ghi nhận trực tiếp như sau:

4.1.

Bảng dữ liệu thực nghiệm thu được

1_DauVan	1	Minimax	(2, 2)	0	25	4.79
		Alpha-Beta	(2, 2)	0	25	3.76
1_DauVan	2	Minimax	(2, 2)	-9600	841	136.23
		Alpha-Beta	(2, 2)	-9600	290	38.77
1_DauVan	3	Minimax	(2, 4)	3200	33865	4665.17
		Alpha-Beta	(2, 4)	3200	4803	645.42

2_GiuaVan	1	Minimax	(2, 2)	-3200	45	7.64
		Alpha-Beta	(2, 2)	-3200	45	6.48
2_GiuaVan	2	Minimax	(0, 1)	-1000000	2155	301.93
		Alpha-Beta	(0, 1)	-1000000	451	57.18
2_GiuaVan	3	Minimax	(0, 1)	-1000001	105363	25562.96
		Alpha-Beta	(0, 1)	-1000001	7389	2065.88

3_AI_SapThang	1	Minimax	(3, 1)	1000000	37	9.46
		Alpha-Beta	(3, 1)	1000000	37	8.58
3_AI_SapThang	2	Minimax	(3, 1)	1000001	1456	378.18
		Alpha-Beta	(3, 1)	1000001	364	95.58
3_AI_SapThang	3	Minimax	(3, 1)	1000002	66776	17279.31
		Alpha-Beta	(3, 1)	1000002	1952	475.54

4_May_Can_Chan	1	Minimax	(4, 2)	0	37	10.18
		Alpha-Beta	(4, 2)	0	37	10.30
4_May_Can_Chan	2	Minimax	(1, 0)	-1000000	1531	402.47
		Alpha-Beta	(1, 0)	-1000000	532	127.49
4_May_Can_Chan	3	Minimax	(1, 0)	-1000001	67081	17808.75
		Alpha-Beta	(1, 0)	-1000001	12431	3251.65

5_HaiBenTanCong	1	Minimax	(5, 5)	43600	41	10.56
		Alpha-Beta	(5, 5)	43600	41	10.33
5_HaiBenTanCong	2	Minimax	(2, 4)	-30800	1794	459.84
		Alpha-Beta	(2, 4)	-30800	604	159.90
5_HaiBenTanCong	3	Minimax	(2, 4)	78800	84954	15082.05
		Alpha-Beta	(2, 4)	78800	13853	1939.28

6_NhieuNhanh	1	Minimax	(2, 6)	1000000	71	10.14
		Alpha-Beta	(2, 6)	1000000	71	11.31
6_NhieuNhanh	2	Minimax	(2, 6)	1000001	4859	706.79
		Alpha-Beta	(2, 6)	1000001	706	103.30
6_NhieuNhanh	3	Minimax	(2, 6)	1000002	333887	76041.98
		Alpha-Beta	(2, 6)	1000002	17031	2527.87

4.2.

Nhận xét và phân tích kết quả định lượng

1. Sự đồng nhất về nước đi và điểm số giữa hai thuật toán

Dữ liệu từ bảng kiểm thử chứng minh rõ ràng: Tại tất cả các trạng thái và các độ sâu, Alpha-Beta luôn trả về chính xác cùng một tọa độ nước đi và cùng giá trị điểm đánh giá với thuật toán Minimax gốc. Ví dụ tại thế cờ 2_GiuaVan độ sâu 3, cả hai đều chọn ô (0, 1) với mức điểm -1000001. Kết quả này khẳng định quá trình cắt nhánh được lập trình hoàn toàn chính xác theo nguyên lý lý thuyết, không bỏ sót bất kỳ nước đi tối ưu nào.

2. Khả năng cắt giảm trạng thái của Alpha-Beta so với Minimax

- Ở độ sâu 1, số lượng trạng thái đã xét của hai thuật toán là trùng nhau do cây tìm kiếm chưa đủ độ sâu để kích hoạt điều kiện cắt nhánh
- Ở độ sâu 2, Alpha-Beta bắt đầu thể hiện sự vượt trội khi cắt giảm được khoảng 65% - 75% lượng trạng thái thừa (Ví dụ ở 1_DauVan, Minimax duyệt 841 trạng thái trong khi Alpha-Beta chỉ cần duyệt 290).
- Ở độ sâu 3, cơ chế cắt nhánh đạt hiệu suất tối đa khi loại bỏ từ 80% đến 97% cây tìm kiếm. Diễn hình ở kịch bản 3_AI_SapThang, Minimax phải tính toán qua 66.776 trạng thái, trong khi Alpha-Beta chỉ cần xét duyệt vốn vẹn 1.952 trạng thái (giảm tải đến 97.07% không gian tìm kiếm).

3. Sự biến động về thời gian chạy khi tăng độ sâu

Thời gian xử lý của cả hai thuật toán tỷ lệ thuận theo hàm mũ đối với độ sâu tìm kiếm.

- Với Minimax, thời gian tăng trưởng mất kiểm soát ở độ sâu 3, lên tới 25.562,96 ms (~ hơn 25 giây) đối với trạng thái 2_GiuaVan, gây ra hiện tượng treo máy/đứng giao diện khi chơi thực tế.
- Với Alpha-Beta, thời gian chạy được tối ưu hóa xuất sắc. Tại thế cờ phức tạp 2_GiuaVan độ sâu 3, Alpha-Beta chỉ tiêu tốn 2.065,88 ms (nhanh gấp 12.3 lần Minimax). Tại thế cờ chứa nước đi quyết định kết thúc sớm như 3_AI_SapThang, Alpha-Beta đưa ra phản hồi chỉ sau 475.54 ms nhờ tìm thấy nước thắng sớm và lập tức đóng băng các nhánh phụ.

4. Ảnh hưởng của độ sâu đến chất lượng nước đi

Độ sâu tìm kiếm quyết định tầm nhìn và trí thông minh của AI.

- Ở độ sâu nông (depth = 1), AI bộc lộ tính chất "cận thị", chỉ biết ăn điểm ngay trước mắt mà không thấy cạm bẫy của đối thủ.
- Ở độ sâu sâu hơn (depth = 3), AI có năng lực phân tích chiến thuật dài hạn. Nó chấp nhận chọn các ô cờ có điểm tĩnh thấp hơn ở lượt hiện tại nhưng tạo ra chuỗi phân phối hình học tối ưu mở rộng (ví dụ dịch chuyển từ ô (2, 2) lên ô chiến lược (2, 4) ở thế cờ đầu ván), giúp chủ động phòng ngự từ xa hoặc ép đối thủ vào thế bị động. Mốc depth = 3 kết hợp cắt nhánh Alpha-Beta là điểm cân bằng lý tưởng nhất giữa tốc độ xử lý phần cứng và độ thông minh của máy tính.

5. ĐÁNH GIÁ ƯU ĐIỂM, HẠN CHẾ VÀ HƯỚNG CẢI TIẾN

5.1.

Trường hợp AI chơi tốt (Ưu điểm)

- Cấu trúc mã nguồn được tổ chức tách biệt module rõ ràng (Modular Design), quản lý hàng số tập trung giúp hệ thống chạy rất ổn định, dễ bảo trì.
- Thuật toán giới hạn bán kính nước đi lân cận giúp kiểm soát tốt tốc độ bùng nổ không gian trạng thái của bàn cờ 9×9 .
- AI xử lý rất sắc bén trong giai đoạn giữa ván và cuối ván (3_AI_SapThang, 4_May_Can_Chan). Nó phát hiện các cơ hội dứt điểm trận đấu cực kỳ nhanh và luôn phản xạ nhảy vào chặn chính xác các chuỗi 3 quân nguy hiểm của đối thủ.

5.2.

Trường hợp AI chơi chưa tốt (Hạn chế chí mạng)

- Hàm Heuristic hiện tại bộc lộ điểm yếu lớn do chỉ tập trung hình phạt nặng cho chuỗi 3 quân của đối thủ (-4000) mà chỉ phạt rất nhẹ đối với chuỗi 2 quân thoáng (-200).
- Dựa trên biến thể luật chơi của đề bài: Chỉ cần 4 quân liên tiếp là thắng và không xét luật chặn hai đầu. Do đó, chỉ cần người chơi con người tạo được một chuỗi 2 quân liên tiếp thoáng (ví dụ . X X .), ở lượt tiếp theo họ sẽ đánh thành hàng 3 thoáng. Lúc này AI chỉ có duy nhất 1 lượt đi nên chỉ chặn được 1 đầu, người chơi chắc chắn sẽ đánh vào đầu trống còn lại ở lượt kế tiếp để đạt 4 quân và giành chiến thắng.
- Vì hàm đánh giá chưa nhận diện được sự nguy hiểm cực độ của chuỗi 2 quân thoáng nên AI thường chủ quan bỏ qua không chặn từ sớm để tập trung đi tấn công ở nơi khác. Đây là nguyên nhân cốt lõi khiến người chơi con người khi đánh trực tiếp luôn luôn dễ dàng thắng máy bằng bất kỳ cờ chuỗi 2 quân thoáng.

5.3.

Hướng cải tiến tiếp theo

Nếu có thêm thời gian phát triển dự án, sinh viên sẽ tập trung nâng cấp các thành phần sau:

1. Cải tiến hàm Heuristic: Tăng mạnh trọng số phạt điểm của chuỗi 2 quân thoáng từ đối thủ (nâng từ -200 lên -3.000 điểm) nhằm ép AI bắt buộc phải chủ động đi chặn đường từ sớm, triệt tiêu mầm mống tạo bất kỳ của con người.
2. Cài đặt Sắp xếp nước đi (Move Ordering): Sắp xếp danh sách các nước đi tiềm năng (các ô nằm sát quân cờ hiện tại, ô tạo chuỗi) lên

duyet trước. Theo lý thuyết cấu trúc dữ liệu, nếu nước đi tốt được duyệt trước, Alpha-Beta sẽ đạt trạng thái cắt nhánh lý tưởng, giúp giảm sâu số trạng thái và nâng độ sâu tìm kiếm lên $depth = 4$ hoặc 5 mà thời gian phản hồi vẫn dưới 1 giây.

6. TÀI LIỆU THAM KHẢO VÀ PHẠM VI SỬ DỤNG

- Slide bài giảng môn Cơ sở Trí tuệ Nhân tạo - Chương "Tìm kiếm có đối thủ" (Thuật toán Minimax & Cắt nhánh Alpha-Beta).
- Tài nguyên mã nguồn mẫu từ GitHub: Repo [husus/gomokuAI-py](#).
- Phần đã tham khảo từ repo mẫu: Tham khảo ý tưởng tổ chức cấu trúc thư mục phân tách module độc lập, cách thức sử dụng ma trận mảng hai chiều để biểu diễn trạng thái bàn cờ và tư duy giới hạn danh sách nước đi khả thi để tối ưu thuật toán. Các thuật toán lõi Minimax/Alpha-Beta, hàm chấm điểm Heuristic chi tiết, bộ kịch bản kiểm thử JSON và toàn bộ phần mã nguồn Caro biến thể 4 quân đều do sinh viên tự cài đặt độc lập, không sao chép nguyên vẹn